

Chapter 7: Strings

Outline

- String as a sequence
- Indexing
- Mutability and Immutability
- Searching
- String Comparison

What is a String?

- a string, the most popular data type in python, is a sequence of characters enclosed within single quotes, or double quotes or triple single quotes or triple double quotes; and can easily be created as such
- When a string contains numbers, it is still a string
- Strings are immutable, meaning once a string is created, it cannot be changed.
- Python provides a wide range of operations and methods for working with strings.

Creating strings

- # Single and double quotes

```
string1 = 'Hello, World!'
```

```
string2 = "Hello, World!"
```

- # Triple quotes for multi-line strings

```
string3 = '''Hello,  
World!'''
```

```
string4 = """Hello,  
World!"""
```

What will print statement on each string output?

Traversing

- Computations on string start at the beginning, select each character in turn, do something to it, and continue until the end.
- This pattern of processing is called a **traversal**.
- The **iteration variable, letter** “iterates” through the **string** and the **block (body)** of code is executed once for each value the iteration variable assumes in the **sequence**
- Traversal for loop:

```
for letter in 'banana':  
    print(letter)
```



- What will be the output?

Looking inside

- We can get at any single character in a string using an index specified in square brackets

```
>>> fruit = 'banana'
```

```
>>> print(fruit[1]) # prints what?
```

Indexing operator

- The expression in brackets is called an index
- The index value must be an integer and starts at zero
- And the largest index is length – 1, where length=len(string at hand)

```
>>> fruit = 'banana'
```

```
>>> letter = fruit[0]
```

```
>>> print(letter) # print b
```

```
>>> length = len(fruit)
```

```
>>> w = fruit[length - 1]
```

```
>>> print(w) #prints a
```

```
>>> print(fruit[-1]) # what will be printed?
```

Indexing

- The first character of a string has index 0, thus the indexing could be 0, 1, 2, ... length-1, where length is the length of the string

```
>>> greeting = 'hello world'
>>> greeting[0]
'h'
>>> 'hello world'[0]
'h'
```

- You can also count back from the end of a string, beginning with -1; the indexing becomes -1, -2, -3, ... (-)length

```
>>> greeting = 'hello, world'
>>> greeting[-1]
'd'
>>> 'hello world'[-1]
'd'
>>> greeting[-11] # same as greeting[0]
'h'
```

Traversal using while loop

```
fruit = 'banana'
```

```
index = 0
```

```
while (index < len(fruit):
```

```
    print(fruit[index])
```

```
    index+=1
```

What will be printed?

Slicing

- A *substring* of a string is obtained by taking a **slice**.
- Need to get continuous section of a string using a **colon operator**.

```
>>> s = 'Monty Python'
```

M	o	n	t	y		P	y	t	h	o	n
---	---	---	---	---	--	---	---	---	---	---	---

```
>>> print(s[0:2])
```

Mo

```
>>> print(s[:2]) #What will be printed?
```

```
>>> print(s[5:]) # What will be printed?
```

Slicing cont'd

- You can use indexes to slice (extract a piece of) a string
- `aStr[i:j]` is the substring that begins with index `i` and ends with (but does not include) index `j`

```
>>> greeting = 'HELLO, WORLD'
>>> greeting[1:3]
'EL'
>>> greeting[-3:-1]
'RL'
```

H	E	L	L	O	,		W	O	R	L	D
0	1	2	3	4	5	6	7	8	9	10	11
-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

- omit begin or end to mean 'as far as you can go'

```
>>> print(greeting[:4], greeting[7:])
HELL WORLD
```

- `aStr[i:j:k]` is the same, but takes only every `k`-th character

```
>>> greeting[3:10:2]#steps by 2, picking every other character
'L,WR'
```

Takeaway exercise: make it step by 3

Slice by step example

- Let's say we have these toy cars in a line:
- Each letter is a different car.
 - Now, how can we pick certain cars from this line?
 - we can use slicing with steps.
- **Starting Point:**
 - Imagine you want to start picking cars from the first car.
 - Your starting point is the first car, which we call car 'A'
- **Ending Point:**
 - You want to stop picking cars before a certain car.
 - Let's say we want to stop before car, 'M'
- **Step:**
 - The step is how many cars you want to skip each time you pick a car.
 - If you want to pick every second car, it means you take one car, skip one, take the next, skip one, and so on.

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12

Implemented:

```
# List of cars as a string  
cars = "ABCDEFGHIJKLM"
```

```
# Slicing with steps to pick every second car from A to L  
selected_cars = cars[0:12:2]
```

```
print(selected_cars)    # Output: "ACEGIK"
```

Takeaway Exercise

```
string3 = '''Hello,
World!'''
string4 = """Hello,
World!"""
print(string3) # what will be printed?
print(string3[:2]) # what will be printed?
print(string3[2:]) # what will be printed?
print(string3[:]) # what will be printed?

index = 0
while index < len(string3):
    print(string3[index], end = "") # what will be printed?
    index += 1
```

A character too far

```
>>>fruit = 'banana'  
>>>print(fruit[6])
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module> IndexError: string index out of range

String concatenation

done using the **+** operator

```
string1 = "Hello"
```

```
string2 = "World"
```

```
combined = string1 + " " + string2
```

```
print(combined) # prints the text "Hello World"
```

done using the **join method**

```
words = ["Hello", "World"] # list
```

```
combined = " ".join(words)
```

```
print(combined)# prints the text "Hello World"
```

Looping and counting

- The following program counts the number of times the letter a appears in a string:

```
word = 'banana'
count = 0
for letter in word:
    if letter == 'a':
        count = count + 1
print(count)
```

- **Takeaway Exercise:** Encapsulate this code in a function named count, and generalize it so that it accepts the string and the letter as arguments.

The in operator

- The keyword **in** is a Boolean operator that takes two strings and returns True if the first appears as a substring in the second:

```
>>> 'a' in 'banana'
```

```
True
```

```
>>> 'seed' in 'banana'
```

```
False
```

String comparison

- The comparison operators work on strings.
- To see if two strings are equal:

```
if word == 'banana':  
    print('All right, bananas.')
```
- Other comparison operations are useful for putting words in alphabetical order:

```
word = input("Please enter your word")  
if word < 'banana':  
    print('Your word,' + word + ', comes before banana.')
```

```
elif word > 'banana':  
    print('Your word,' + word + ', comes after banana.')
```

```
else: print('All right, bananas.')
```

String comparison cont'd

- Note that Python is case sensitive.
- All the uppercase letters come before all the lowercase letters, so: Your word, Pineapple, comes before banana.
- A common way to address this problem is to convert strings to a standard format, such as all lowercase, before performing the comparison.
- Takeaway question: How do you convert a string into all lowercase?

string methods

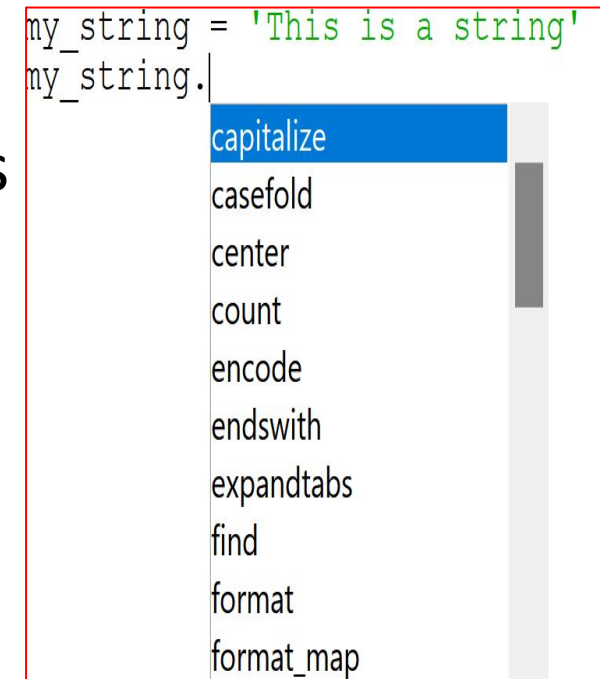
- Strings are an example of Python objects.
- An object contains both data (the actual string itself) and methods, which are effectively functions that are built into the object and are available to any instance of the object.
- Python has a function called **dir** which lists the methods available for an object.
- The type function shows the type of an object and the dir function shows the available methods (next slide)

String methods

- ['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']

String commands cont'd

- You can access the string methods by dot operator
- your editor might pop up a selection window
 - typically by pressing Tab — showing all the methods



The screenshot shows a code editor with the following text: `my_string = 'This is a string'` on the first line and `my_string.` on the second line. A dropdown menu is open below the second line, displaying a list of string methods: `capitalize`, `casefold`, `center`, `count`, `encode`, `endswith`, `expandtabs`, `find`, `format`, and `format_map`. The `capitalize` method is highlighted with a blue background. A vertical scrollbar is visible on the right side of the dropdown menu.

String Methods cont'd

- A method is a function that is bundled together with a particular type of object
- A string method is a function that works on a string
- This is the syntax of a method:

```
anObject.methodName(parameterList)
```

- For example,

```
>>> 'avocado'.index('a')  
0
```

- returns the index of the first 'a' in 'avocado'
- You can also use a variable of type string

```
>>> fruit = 'avocado'  
>>> fruit.index('a')  
0
```

String Methods cont'd

- Python has many very useful string methods
- You should always look for and use an existing string method before coding it again for yourself. Here are some

```
s.capitalize()  
s.count()  
s.endswith() / s.startswith()  
s.find() / s.index()  
s.format()  
s.isalpha() / s.isdigit() / s.islower() / s.isspace()  
s.join()  
s.lower() / s.upper()  
s.replace()  
s.split()  
s.strip()
```

The split() method

- The string method split() lets us separate a string into useful parts
 - Common use: splitting a sentence into its words
- Splits by space character by default,

```
>>> my_string = "Captain, incoming transmission!"
>>> print(my_string.split())
['Captain,', 'incoming', 'transmission!']
>>>
```

- but you can give it a different 'separator' string

```
>>> my_string = "Captain, incoming transmission!"
>>> print(my_string.split(','))
['Captain', ' incoming transmission!']
>>>
```

```
>>> my_string = "a one, a two, a one two three four"
>>> print(my_string.split(','))
['a one', ' a two', ' a one two three four']
>>>
```

The strip() method

- The string method strip() “cleans” the edges of a string by removing the character(s) you specify (default: spaces)

```
>>> my_string = "Hello world! any parenthesis ()"  
>>> print(my_string.strip('! ()'))  
Hello world! any parenthesis  
>>>
```

- Takeaway question: Why is the “!” not removed in the above output?
- The string module contains a useful **variable** for this, called punctuation (like how the math module has pi)

```
>>> import string  
>>> string.punctuation  
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'  
>>>
```

The split() and strip() together

- The split method is useful for extracting words, and the strip method is useful for cleaning them up
- Remember that strip() is a string method, **not** a list method

```
>>> import string
>>> words = ['How', 'are', 'you,', 'sir?']
...
>>> print(words.strip(string.punctuation))
...
Traceback (most recent call last):
  File "<pyshell#19>", line 1, in <module>
    print(words.strip(string.punctuation))
AttributeError: 'list' object has no attribute 'strip'
>>>
```

- So, how can we clean up every word in a sentence, once we've split it?

split() and strip() together cont'd

- The strip() method works on one “word” at a time
- So, take it one word at a time

```
import string
words = ["It's", 'rainy', 'today!', "Isn't", 'it?']
for item in words:
    print(item.strip(string.punctuation))
```

- Here items in the list are string each
- Takeaway question: why can't we just use the replace() method to get rid of punctuation like this?

Method parameters

- Like any function, a method has zero or more parameters
- Even if the parameter list is empty, the method still works on the 'calling' object:

```
>>> my_string = "Hello World"
...
>>> my_string.isupper()
...
False
>>> my_string.isupper() #returns True if tjtat refered to by my_string is ALL CAP
...
False
>>> my_string.isupper() #returns True if All Caps
...
False
>>> my_string.islower() #returns True if all are in lower case
...
False
>>> my_string.isalpha() #returns True if all are alphabets
...
False
>>>
```

Method parameters cont'd

- The `isalpha()` method in python returns 'True' if all the characters in a string are alphabetic and there is at least one character in the string. Otherwise, it returns 'False'
- Alphabetic characters are those defined in the **Unicode character** database as "Letter," including both uppercase and lowercase letters.

```
# Ethiopic characters without and with space
ethiopic_string = "አማርኛ" # Ethiopic word
# Check if the string is alphabetic
print(ethiopic_string.isalpha()) # Output: True

ethiopic_string = "አማርኛ እና ትግርኛ" # Ethiopic word with space
# Check if the string is alphabetic
print(ethiopic_string.isalpha()) # Output: False
```

Method parameters cont'd

- The `replace()` method takes two or three parameters: mandatory: 'old' and 'new'; and 'count' (optional)
 - If 'count' is not provided, all occurrences of 'old' will be replaced by 'new'
 - If 'count' is provided, only the first 'count' occurrences of 'old' will be replaced with 'new'

- Example:

```
my_string = "I saw a saw cutting a saw. I never saw such a saw."
#replace all the 'saw' with a 'sow', do this without 'count' parameter
print(my_string)
print(my_string.replace('saw', 'sow'))#output?
#replace the first two occurrences of 'saw' with 'SOW' (Statement of Work)
print(my_string.replace('saw', 'SOW', 2))#output?
```

- How to replace only the second and the fifth occurrence of 'saw' with 'SOW', to output: **I saw a SOW cutting a saw. I never saw such a SOW!**
- (next slide)

Check it!

```
# Original string
my_string = "I saw a saw cutting a saw. I never saw such a saw!"

# Word to replace
old_w = "saw"
new_w = "SOW"

# Function to replace specific occurrences
def replace_specific_occurrences(a_string, old, new, occurs):
    items = my_string.split(old) #create list of string using separator 'old'
    result = items[0] #initialize with the first element of the split string
    count = 0

    for i in range(1, len(items)): #list starting from the second element
        count += 1 # keeping a count of the occurrences
        if count in occurs: #selective replacement
            result += new + items[i]
        else:
            result += old + items[i]

    return result

# Replace the second and fifth occurrences
occurrences = [2, 5] # 1-based index for occurrences
my_result = replace_specific_occurrences(my_string, old_w, new_w, occurrences)

print(my_result) # Output: I saw a SOW cutting a saw. I never saw such a SOW!
```

Strings are immutable

- A string is immutable -- once created it can not be modified
- Any operation that appears to modify a string actually creates a new string; the original string is not changed

```
>>> aStr = 'my cat is catatonic'
>>> newStr = aStr.replace('cat', 'dog')
>>> newStr
'my dog is dogatonic'
>>> aStr
'my cat is catatonic'
```

- However, you can associate the old string name with the new object

```
>>> aStr = 'my cat is catatonic'
>>> aStr = aStr.replace('cat', 'dog')
>>> aStr
'my dog is dogatonic'
```

Strings and the print () function

- The print() function always prints a string. The input() function always inputs a string.
- Every object in Python has a string representation. Therefore, every object can be printed.
- When you print a number, a list or a function it is the string representation of the object that is printed
- print() takes 0 or more arguments and prints their string representations, separated by spaces

```
>>> print('pi =', 3.14)
pi = 3.14
>>>
```

```
>>> def f():
...     pass
...
>>> print(f)
<function f at 0x000001CFF248FB00>
>>>
```

The print separator and end

- By default, print() separates multiple outputs with spaces

```
>>> print(1, 2, 3)
1 2 3
>>>
```

- You can change this to any string that you want, for example, a colon and a space (': ')

```
>>> print(1, 2, 3, sep=': ')
1: 2: 3
>>>
```

- By default, print() ends its output with a newline ('\n')

```
>>> for i in range(3):
...     print(i)
...
0
1
2
```

- You can change this, for example, to a hyphen or a white space

```
>>> for i in range(3):
...     print(i, end = '-')
...
0-1-2-
>>> for i in range(3):
...     print(i, end = ' ')
...
0 1 2
>>>
```

The string format() method

- The format() method allows you to create formatted strings by embedding expressions inside string literals, using curly braces {} as placeholder and allows flexible formatting options
- format() can take multiple arguments, which are inserted into the placeholders in the string

```
name = "Selamawit"
year = "fifth"
cgpa = 3.95
formatted_string = "My name is {} and I am a {} year software engineering student with {} cumulative GPA.".format(name, year, cgpa)
print(formatted_string) # Output: "My name is Selamawit and I am a fifth year software engineering student with 3.95 cumulative GPA."
```

The **in** and **not in** operators

- The **in** operator tests for membership. When both of the arguments to **in** are strings, **in** checks whether the left argument is a substring of the right argument

```
>>> "p" in "apple"  
True  
>>> "i" in "apple"  
False
```

```
>>> "ap" in "apple"  
True  
>>> "pa" in "apple"  
False
```

- Note that a string is a substring of itself, and the empty string is a substring of any other string.

```
>>> "apple" in "apple"  
True  
>>> "" in "a"  
True
```

The **in** and **not in** operators cont'd

- The not in operator returns the logical opposite results of in:
-

```
>>> "x" not in "apple"  
True
```

Takeaway Exercises

1. Combining the **in** operator with string concatenation using **+**, and write a function that removes all the vowels from a string
2. What is the result of each

```
>>> "Python"[1]
>>> "Strings are sequences of characters."[5]
>>> len("wonderful")
>>> "Mystery"[:4]
>>> "p" in "Pineapple"
>>> "apple" in "Pineapple"
>>> "pear" not in "Pineapple"
>>> "apple" > "pineapple"
>>> "pineapple" < "Peach"
```

3. Write a function that recognizes palindromes.

Takeaway Exercises cont'd

1. Write a function that counts how many times a substring occurs in a string which responds as follows when called"

```
count("is", "Mississippi") == 2
count("an", "banana") == 2
count("ana", "banana") == 2
count("nana", "banana") == 1
count("nanan", "banana") == 0
count("aaa", "aaaaaa") == 4
```

2. Write a function that removes the first occurrence of a string from another string, responding as follows when called:

```
remove_all("an", "banana") == "ba"
remove_all("cyc", "bicycle") == "bile"
remove_all("iss", "Mississippi") == "Mippi"
remove_all("eggs", "bicycle") == "bicycle"
```

Takeaway Exercises cont'd

1. Take the following Python code that stores a string:

```
str = "X-DSPAM-Confidence:0.8475"
```

Use find and string slicing to extract the portion of the string after the colon character and then use the float function to convert the extracted string into a floating point number.

2. Explore the built in functions defined in the string module

Takeaway Exercises cont'd

1. Write a Python program to print the index of a character in a string.

Sample string: w3resource

Expected output:

Current character w position at 0

Current character 3 position at 1

Current character r position at 2

2. Explore string tutorial and exercises on:

https://www.w3schools.com/python/python_strings.asp

End of Chapter 7